

✓ Fall 2019 CX4641/CS7641 Homework 2

Instructor: Dr. Mahdi Roozbahani

Deadline: Oct 8, Tuesday, 11:59 pm

- No unapproved extension of the deadline is allowed. Late submission will lead to 0 credit.
- Discussion is encouraged, but each student must write his own answers and explicitly mention any collaborators.
- Homework submission ONLY in .ipynb format.

Instructions for the assignment

In this assignment, we have both coding and writing questions: you are asked to code and answer them in the cells.

- Graduate students are required to complete all the questions including **bonus parts**. Undergraduate students are welcome to try bonus questions and we will add them on your final grade.
- To switch between cell for code and for markdown, see the menu -> Cell -> Cell Type
- You could directly type the Latex equations in the markdown cell.
- Typing with Latex is highly recommended. An image scan copy of handwritten would work if and only if you upload the images to a public cloud or github and refer them within your ipython notebook using the created url.

✓ 0 Set up

This notebook is tested under [python 3.6.8](#), and the corresponding packages can be downloaded from [miniconda](#). You may also want to get yourself familiar with several packages:

- [jupyter notebook](#)
- [numpy](#)
- [matplotlib](#)

Please implement the functions that has "raise NotImplementedError", and after you finishing the coding, please delete or comment "raise NotImplementedError".

```
[ ] #####
### DO NOT CHANGE THIS CELL ###
#####

from __future__ import absolute_import
from __future__ import print_function
from __future__ import division

%matplotlib inline

import sys
import matplotlib
import numpy as np
import matplotlib.pyplot as plt
import imageio
from tqdm import tqdm_notebook as tqdm

print('Version information')

print('python: {}'.format(sys.version))
print('matplotlib: {}'.format(matplotlib.__version__))
print('numpy: {}'.format(np.__version__))

# Set random seed so output is all same
```

```
np.random.seed(1)

# url for image in KMeans
url = 'https://zbigatron.com/wp-content/uploads/2019/09/grumpy_cat.jpg'
```

Version information
python: 3.7.2 (default, Dec 29 2018, 00:00:04)
[Clang 4.0.1 (tags/RELEASE_401/final)]
matplotlib: 3.0.2
numpy: 1.15.4

1. KMeans Clustering [25 + 5pts]

KMeans is trying to solve the following optimization problem:

$$\arg \min_S \sum_{i=1}^K \sum_{x_j \in S_i} ||x_j - \mu_i||^2$$

where one needs to partition the N observations into K sets: $S = \{S_1, S_2, \dots, S_K\}$ and each set has μ_i as its center.

1.1 pairwise distance [5pts]

Given $X \in \mathbb{R}^{N \times D}$ and $Y \in \mathbb{R}^{M \times D}$, obtain the pairwise distance matrix $dist \in \mathbb{R}^{N \times M}$ using the euclidean distance metric, where $dist_{i,j} = ||X_i - Y_j||_2$.

DO NOT USE A FOR LOOP in your implementation -- they are slow and will make your code too slow to pass our grader. Use array broadcasting instead.

```
def pairwise_dist(x, y):
    """
    Args:
        x: N x D numpy array
        y: M x D numpy array
    Return:
        dist: N x M array, where dist[i, j] is the euclidean distance between
        x[i, :] and y[j, :]
    """

    dist = np.sqrt((np.square(x[:, np.newaxis] - y).sum(axis=2)))
    return dist
```

```
# TEST CASE
x = np.random.randn(2, 2)
y = np.random.randn(3, 2)

print("*** Expected Answer ***")
print("====x====")
[[ 1.62434536 -0.61175641]
 [-0.52817175 -1.07296862]]
==y==
[[ 0.86540763 -2.3015387 ]
 [ 1.74481176 -0.7612069 ]
 [ 0.3190391  -0.24937038]]
==dist==
[[1.85239052 0.19195729 1.35467638]
 [1.85780729 2.29426447 1.18155842]]")

print("\n*** My Answer ***")
print("====x====")
print(x)
print("====y====")
print(y)
print("====dist====")
print(pairwise_dist(x, y))
```

```
*** Expected Answer ***
====x====
[[ 1.62434536 -0.61175641]
 [-0.52817175 -1.07296862]]
==y==
[[ 0.86540763 -2.3015387 ]
 [ 1.74481176 -0.7612069 ]
 [ 0.3190391  -0.24937038]]
```

```

==dist==
[[1.85239052 0.19195729 1.35467638]
 [1.85780729 2.29426447 1.18155842]]

*** My Answer ***
==x==
[[ 1.62434536 -0.61175641]
 [-0.52817175 -1.07296862]]
==y==
[[ 0.86540763 -2.3015387 ]
 [ 1.74481176 -0.7612069 ]
 [ 0.3190391  -0.24937038]]
==dist==
[[1.85239052 0.19195729 1.35467638]
 [1.85780729 2.29426447 1.18155842]]

```

1.2 KMeans Implementation [20pts]

In this section, you are asked to implement the kmeans algorithm. Below is a template which requires you to implement some member functions that haven't been implemented yet.

For the function signature, please see the corresponding doc strings.

```

[ ] class KMeans(object):

    def __init__(self): #No need to implement
        pass

    def _init_centers(self, points, K, **kwargs):
        """
        Args:
            points: NxD numpy array, where N is # points and D is the dimensionality
            K: number of clusters
            kwargs: any additional arguments you want
        Return:
            centers: K x D numpy array, the centers.
        """
        self.K = K
        centers = points[np.random.choice(points.shape[0], self.K, replace=False)]

        return centers

    def _update_assignment(self, centers, points):
        """
        Args:
            centers: KxD numpy array, where K is the number of clusters, and D is the dimension
            points: NxD numpy array, the observations
        Return:
            cluster_idx: numpy array of length N, the cluster assignment for each point

        Hint: You could call pairwise_dist() function.
        """
        N = points.shape[0]
        cluster_idx = np.zeros((N, 1), dtype=int)
        dist_mat = pairwise_dist(points, centers)

        cluster_idx = np.argmin(dist_mat, axis=1)

        cluster_idx.reshape(cluster_idx.shape[0],1)

        return cluster_idx

    def _update_centers(self, old_centers, cluster_idx, points):
        """
        Args:
            old_centers: old centers KxD numpy array, where K is the number of clusters, and D is the dimension
            cluster_idx: numpy array of length N, the cluster assignment for each point
            points: NxD numpy array, the observations
        Return:
            centers: new centers, K x D numpy array, where K is the number of clusters, and D is the dimension.
        Note:
            It is possible to have fewer centers after this step.
        """
        no_cluster = old_centers.shape[0]
        D = old_centers.shape[1]
        centers = np.zeros((no_cluster, D), dtype=int)

```

```

for i in range(0, no_cluster):
    a = np.where((cluster_idx[:, None] == i).all(-1))
    centers[i] = points[a].mean(axis=0)

return centers

def _get_loss(self, centers, cluster_idx, points):
    """
    Args:
        centers: KxD numpy array, where K is the number of clusters, and D is the dimension
        cluster_idx: numpy array of length N, the cluster assignment for each point
        points: NxD numpy array, the observations
    Return:
        loss: a single float number, which is the objective function of KMeans.
    """
    K = centers.shape[0]
    distance = np.zeros(points.shape[0])

    for k in range(0, K):
        distance[cluster_idx == k] = np.linalg.norm(points[cluster_idx == k] - centers[k], axis=1)
    return np.sum(np.square(distance))

def __call__(self, points, K, max_iters=100, abs_tol=1e-16, rel_tol=1e-16, verbose=False, **kwargs):
    """
    Args:
        points: NxD numpy array, where N is # points and D is the dimensionality
        K: number of clusters
        max_iters: maximum number of iterations (Hint: You could change it when debugging)
        abs_tol: convergence criteria w.r.t absolute change of loss
        rel_tol: convergence criteria w.r.t relative change of loss
        verbose: boolean to set whether method should print loss (Hint: helpful for debugging)
        kwargs: any additional arguments you want
    Return:
        cluster assignments: Nx1 int numpy array
        cluster centers: K x D numpy array, the centers
        loss: final loss value of the objective function of KMeans
    """
    centers = self._init_centers(points, K, **kwargs)
    for it in range(max_iters):
        cluster_idx = self._update_assignment(centers, points)
        centers = self._update_centers(centers, cluster_idx, points)
        loss = self._get_loss(centers, cluster_idx, points)
        K = centers.shape[0]
        if it:
            diff = np.abs(prev_loss - loss)
            if diff < abs_tol and diff / prev_loss < rel_tol:
                break
        prev_loss = loss
        if verbose:
            print('iter %d, loss: %.4f' % (it, loss))
    return cluster_idx, centers, loss

```

1.3 Pixel Clustering

In this section, you will use your K-Means algorithm to do pixel clustering which compresses the image using K colors.

```

[ ] # helper function for plotting images. You don't have to modify it

def plot_images(img_list, title_list, figsize=(11, 6)):
    assert len(img_list) == len(title_list)
    fig, axes = plt.subplots(1, len(title_list), figsize=figsize)
    for i, ax in enumerate(axes):
        ax.imshow(img_list[i] / 255.0)
        ax.set_title(title_list[i])
        ax.axis('off')

```

```

[ ] # helper function for plotting images. You don't have to modify it

def cluster_pixels_kmeans(image, K):
    """Clusters pixels in the input image

    Args:
        image: input image of shape(H, W, 3)
        K: number of clusters

```



```

Return:
    clustered_img: image of shape(H, W, 3) after pixel clustering
"""
im_height, im_width, im_channel = image.shape
flat_img = np.reshape(image, [-1, im_channel]).astype(np.float32)
cluster_ids, centers, loss = KMeans()(flat_img, K)
kmeans_img = np.reshape(centers[cluster_ids], (im_height, im_width, im_channel))
return kmeans_img

```

```

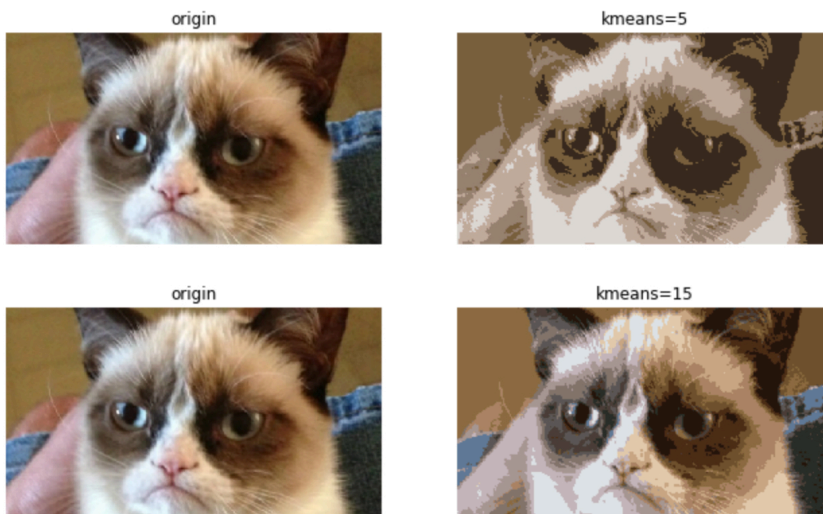
[ ] # helper function for plotting images. You don't have to modify it

image = imageio.imread(imageio.core.urlopen(url).read())

kmeans_image_5 = cluster_pixels_kmeans(image, 5)
plot_images([image, kmeans_image_5], ['origin', 'kmeans=5'])

kmeans_image_15 = cluster_pixels_kmeans(image, 15)
plot_images([image, kmeans_image_15], ['origin', 'kmeans=15'])

```



1.4 (Bonus) Find the optimal number of clusters [5pts]

You will now use the elbow method to find the optimal number of clusters to use for pixel clustering the given image.

```

[ ] def find_optimal_num_clusters(image, max_K=10):
    """Plots loss values for different number of clusters in K-Means

    Args:
        image: input image of shape(H, W, 3)
        max_K: number of clusters

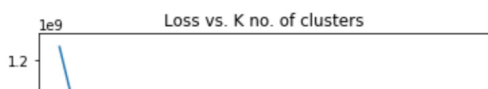
    Return:
        None (plot loss values against number of clusters)
    """

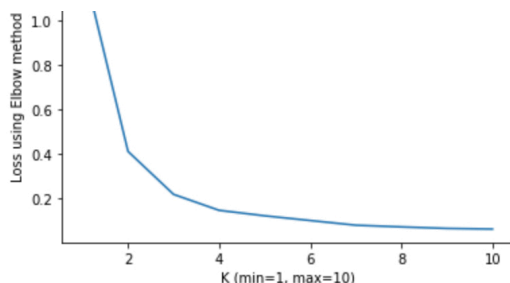
    loss_list = []
    k_list = []
    im_height, im_width, im_channel = image.shape
    for k in range(1, max_K + 1):
        flat_img = np.reshape(image, [-1, im_channel]).astype(np.float32)
        cluster_ids, centers, loss = KMeans()(flat_img, k)
        loss_list.append(loss)
        k_list.append(k)

    plt.plot(k_list, loss_list)
    plt.title('Loss vs. K no. of clusters')
    plt.xlabel('K (min=1, max=10)')
    plt.ylabel('Loss using Elbow method')

    find_optimal_num_clusters(image)

```





✓ 2 EM algorithm [20 pts]

EM algorithm is a widely used approach to learning in the presence of unobserved variables. Consider the general framework of the EM algorithm, given a joint distribution $P(x, z|\theta)$ over observed variables x , hidden variable z and its distribution $q(z)$, and governing parameter θ , the goal is to maximize the likelihood function $P(x|\theta)$ and given the following expression:

$$\log P(x|\theta) = \log\left(\sum_z P(x, z|\theta)\right) = \log\left(\sum_z q(z) \frac{P(x, z|\theta)}{q(z)}\right) \geq \sum_z q(z) \log \frac{P(x, z|\theta)}{q(z)} = \sum_z q(z) \log \frac{P(z|x, \theta)P(x|\theta)}{q(z)}$$

The inequality is based on the Jensen's theorem.

- Please provide a brief description on how to perform the E step in the above equation.
- Please provide a brief description on how to perform the M step in the above equation.

Suppose

$$F(q, \theta) = \sum_z q(z) \log \frac{P(x, z|\theta)}{q(z)}$$

During the lecture, the expression $\sum_z q(z) \log \frac{P(x, z|\theta)}{q(z)}$ was explained through the sum of entropy and log-likelihood; the same approach can also be explained using KL-divergence.

- Please derive that from the above equation (**Hint:** use product rule for joint probability) and explain the what will happen to the KL term in the E step.

Explanation:

First we want to initialize parameters θ^{old} .

E-step: Let $q(z) = p(z|x, \theta^{old})$. Let $F(q, \theta^{old}) = \sum_z q(z) \log \frac{P(x, z|\theta)}{q(z)}$.

M-step: Based on expectation value, calculate $\theta^{new} = \underset{\theta}{\operatorname{argmax}} F(q, \theta^{old})$.

Iterate for θ until convergence of params or likelihood is reached, or when maximum steps are reached.

Derivation:

From above we know that $F(q, \theta)$ is the lower bound.

$$F(q, \theta) = \sum_z q(z) \log \frac{P(x, z|\theta)}{q(z)}$$

Using product rule:

$$\begin{aligned} &= \sum_z q(z) \log \frac{P(z|x, \theta)P(x|\theta)}{q(z)} \\ &= \sum_z q(z) \log \frac{P(z|x, \theta)}{q(z)} + \sum_z q(z) \log P(x|\theta) \\ &= -KL[q(z), P(z|x, \theta)] + \log P(x|\theta) \end{aligned}$$

(By definition, $KL(q, p) = \sum_z q(z) \log \frac{q(z)}{p(z)}$)

Since $\log P(x|\theta)$ is the log-likelihood we want, we can calculate $\log P(x|\theta)$ by calculating $F(q, \theta) + KL[q(z), P(z|x, \theta)]$. $\square$

3. GMM implementation [30 pts]

GMM is trying to do MLE (maximum likelihood estimation). It approximates the distribution of data using a set of gaussian distributions. Here we assume that each gaussian component has diagonal covariance matrix, which makes it easier to calculate the density.

Given N samples $X = [x_1, x_2, \dots, x_N]$, we are asked to find K diagonal gaussian distributions to model the data X :

$$\max_{\{\mu_k, \sigma_k\}_{k=1}^K} \sum_{i=1}^N \log \left(\sum_{k=1}^K \pi(k) \mathcal{N}(x_i | \mu_k, \sigma_k) \right)$$

- For undergraduate student: you may assume the covariance matrix is diag matrix, which means the features are independent. (i.e. the red intensity of a pixel is independent from its blue intensity, etc).
- For graduate student: please assume full covariance matrix.

Hints

1. Here $\pi(\cdot)$ is the prior of the latent variable. It is also called the mixture coefficient. To make it simple, we assume $\pi(k) = \frac{1}{K}, \forall k = 1, 2, \dots, K$.
2. As we create our model, we will need to use a multivariate Gaussian since our pixels are 3-dimensional vectors corresponding to red, green, and blue color intensities. It means that for each image, you need to convert it into a $N \times 3$ matrix, where N is the number of pixels, and 3 is the number of features.

The following example from a machine learning textbook may be helpful:

3. In this question, each pixel has three features, which are R, G, and B.
4. At EM steps, gamma means $\tau(z_{nk})$ at our slide of GMM, which is called the responsibility. If we have K components, each data point (pixel) will have K responsibility values.
5. For E steps, we already get the log-likelihood at `ll_joint()` function. For the fomula at our slide:

$$\tau(z_{nk}) = \frac{\pi_k N(x_n | \mu_k, \Sigma_k)}{\sum_{j=1}^K \pi_j N(x_n | \mu_j, \Sigma_j)},$$

`ll_joint` equals to the N here. Thus, You should be able to finish E steps with just a few lines of code by using `ll_joint()` and `softmax()` defined above.

3.1 Helper functions

softmax [5 pts]

Given $logit \in \mathbb{R}^{N \times D}$, calculate $prob \in \mathbb{R}^{N \times D}$, where $prob_{i,j} = \frac{\exp(logit_{i,j})}{\sum_{k=1}^D \exp(logit_{i,k})}$.

Note that it is possible that $logit_{i,j}$ is very large, making $\exp(\cdot)$ of it to explode. To make sure it is numerical stable, you may need to subtract the maximum for each row of *logits*. As in calculating pairwise distances, DO NOT USE A FOR LOOP.

[1]

```
def softmax(logits):
    """
    Args:
        logits: N x D numpy array
    """

    row_max = np.amax(logits, 1)
    col_no = np.size(logits, 1)
    diff = np.subtract(logits, row_max.reshape(col_no, 1))
    exp = np.exp(diff)
    row_sum = exp.sum(axis=1).reshape(col_no, 1)
    x = exp / row_sum

    return x

logits = np.array([[1000, 1000],
                  [1, 2]], dtype = np.float32)

print("""Correct answer:
===softmax===
[[0.5      0.5      ]
 [0.26894143 0.7310586 ]]""")

print("My answer:")
print(softmax(logits))
```

✓

```
Correct answer:
===softmax===
```

```
[[0.5      0.5      ]
 [0.26894143 0.7310586 ]]
My answer:
[[0.5      0.5      ]
 [0.26894143 0.7310586 ]]
```

✓ logsumexp [5 pts]

Given $\text{logit} \in \mathbb{R}^{N \times D}$, calculate $s \in \mathbb{R}^N$, where $s_i = \log \left(\sum_{j=1}^D \exp(\text{logit}_{i,j}) \right)$. Again, pay attention to the numerical problem. You may want to use similar trick as in the softmax function. DO NOT USE A FOR LOOP.

```
[ ]
def logsumexp(logits):
    """
    Args:
        logits: N x D numpy array
    Return:
        s: N x 1 array where s[i,0] = logsumexp(logits[i,:])
    """

    row_max = np.amax(logits, 1)
    col_no = np.size(logits, 1)
    diff = np.subtract(logits, row_max.reshape(col_no, 1))
    exp = np.exp(diff)
    row_sum = exp.sum(axis=1).reshape(col_no, 1)
    log_x = np.log(row_sum) + row_max.reshape(col_no, 1)

    return log_x

logits = np.array([[1000, 1000],
                  [1, 2]], dtype=np.float32)

print("""Correct Answer:
===logsumexp===
[[1000.6932  ]
 [  2.3132617]]""")

print("My answer: ")
print(logsumexp(logits))
```

✓

```
Correct Answer:
===logsumexp===
[[1000.6932  ]
 [  2.3132617]]
My answer:
[[1000.6932  ]
 [  2.3132617]]
```

✓ 3.2 GMM Implementations [20pts]

```
[ ]
class GMM(object):
    def __init__(self): # No need to implement
        pass

    def _init_components(self, points, K, **kwargs):
        """
        Args:
            points: NxD numpy array, the observations
            K: number of components
            kwargs: any other args you want
        Return:
            pi: numpy array of length K, prior
            mu: KxD numpy array, the center for each gaussian.
            sigma: KxDxD numpy array, the covariance matrix of each gaussian.

        Hint: You could use the K-means results to initial GMM. It will help to converge.
        For instance, you could use ids, mu, ~ = KMeans()(points, K) to initialize.
        """
        self.K = K
        self.D = points.shape[1]

        cluster_id, mu, _ = KMeans()(points, self.K)

        self.initial_pi = np.zeros(self.D)
```

```

self.initial_means = np.zeros((self.K, self.D))
self.initial_cov = np.zeros((self.K, self.D, self.D))

for i in range(0, self.K):
    idx = np.where(cluster_id == i) # returns indices of points which belongs to cluster i (= 0 to k-1)
    self.initial_pi[i] = len(idx[0]) / points.shape[0] # no. of points belongs to cluster i / total pair
    self.initial_means[i] = np.mean(points[idx], axis=0) # mean of points belonging to cluster i
    de_meaned = points[idx] - self.initial_means[i,:] # (x - mu)
    Nk = points[idx].shape[0] # seems to be same as len(idx[0])
    self.initial_cov[i,:,:] = np.dot(self.initial_pi[i] * de_meaned.T, de_meaned) / Nk # (x-mu)(x-mu).T

#     assert np.sum(self.initial_pi) == 1

pi = self.initial_pi
mu = self.initial_means
sigma = self.initial_cov
return (pi, mu, sigma)

def _ll_joint(self, points, pi, mu, sigma, **kwargs):
    """
    Args:
        points: NxD numpy array, the observations
        pi: np array of length K, the prior of each component
        mu: KxD numpy array, the center for each gaussian.
        sigma: KxDxD numpy array, the covariance matrix of each gaussian.
    Return:
        ll(log-likelihood): NxK array
    """
    # calculate tau
    N = points.shape[0]
    self.tau = np.zeros((N, self.K))

    for i in range(0, self.K):
        de_mean_2 = points - mu[i]
        det = np.linalg.det(sigma)
        denom_const = 1.0/ ((2 * np.pi) ** N * det) ** (1.0 / 2)
        points_mu = points - mu[i]
        result = np.exp( -0.5 * (np.matmul(points_mu.T, points_mu)/sigma))
        mvn = denom_const * result

        ll = np.sum(logsumexp(pi[i] * mvn))

    return ll

def _E_step(self, points, pi, mu, sigma, **kwargs):
    """
    Args:
        points: NxD numpy array, the observations
        pi: np array of length K, the prior of each component
        mu: KxD numpy array, the center for each gaussian.
        sigma: KxDxD numpy array, the covariance matrix of each gaussian.
    Return:
        gamma: NxK array, the posterior distribution (a.k.a, the soft cluster assignment) for each observati

    Hint: You should be able to do this with just a few lines of code by using _ll_joint() and softmax() def
    """

    N = points.shape[0]
    gamma = np.zeros(N, self.K)

    ll = _ll_joint(points, pi, mu, sigma)

    gamma = softmax (self.pi[i] * ll)

    # normalize across columns
    gamma_norm = np.sum(gamma, axis=1)[:,np.newaxis]
    gamma /= gamma_norm

    return gamma

def _M_step(self, points, gamma, **kwargs):
    """
    Args:

```


points: NxD numpy array, the observations
gamma: NxK array, the posterior distribution (a.k.a, the soft cluster assignment) for each observati
Return:
pi: np array of length K, the prior of each component
mu: KxD numpy array, the center for each gaussian.
sigma: KxDxD numpy array, the covariance matrix of each gaussian.

Hint: There are formulas in the slide.

"""

N = points.shape[0]

D = points.shape[1]

K = gamma.shape[1]

for k in range(0, K):

pi = np.mean(gamma[:,k], axis=0)

mu = np.dot(gamma.T, points) / np.sum(gamma[k], axis = 0)[: ,np.newaxis]

x = points - mu[k, :] # (N x d)

gamma_diag = np.diag(gamma[:, k])

x_mean = np.matrix(x)

gamma_diag = np.matrix(gamma_diag)

sigma_k = x.T * gamma_diag * x

sigma[k,:,:]=(sigma_k) / np.sum(gamma, axis = 0)[: ,np.newaxis][c]

return pi, mu, sigma

def __call__(self, points, K, max_iters=100, abs_tol=1e-16, rel_tol=1e-16, **kwargs):

"""

Args:

points: NxD numpy array, where N is # points and D is the dimensionality

K: number of clusters

max_iters: maximum number of iterations

abs_tol: convergence criteria w.r.t absolute change of loss

rel_tol: convergence criteria w.r.t relative change of loss

kwargs: any additional arguments you want

Return:

gamma: NxK array, the posterior distribution (a.k.a, the soft cluster assignment) for each observati

(pi, mu, sigma): (1xK np array, KxD numpy array, KxD numpy array), mu and sigma.

Hint: You do not need to change it. For each iteration, we process E and M steps, then

"""

pi, mu, sigma = self._init_components(points, K, **kwargs)

pbar = tqdm(range(max_iters))

for it in pbar:

E-step

gamma = self._E_step(points, pi, mu, sigma)

M-step

pi, mu, sigma = self._M_step(points, gamma)

calculate the negative log-likelihood of observation

joint_ll = self._ll_joint(points, pi, mu, sigma)

loss = -np.sum(logsumexp(joint_ll))

if it:

diff = np.abs(prev_loss - loss)

if diff < abs_tol and diff / prev_loss < rel_tol:

break

prev_loss = loss

pbar.set_description('iter %d, loss: %.4f' % (it, loss))

return gamma, (pi, mu, sigma)

[1

```
y = np.array([[1,6,7],
               [3,4,4],
               [5,9,6],
               [2,6,3],
               [1,1,4],
               [5,5,8]])
```

```
gmm = GMM()
result = gmm._init_components(y, 3)
#print(result)
```



```
gmm._ll_joint(y,result[0], result[1], result[2])
```

```
y[:,0]
```

```
[[[nan inf nan]
  [nan inf inf]
  [nan inf inf]]
```

```
[[nan nan nan]
  [nan nan nan]
  [nan nan nan]]
```

```
[[nan inf inf]
  [nan inf inf]
  [nan inf inf]]]
```

```
/Users/anaconda3/lib/python3.7/site-packages/ipykernel_launcher.py:64: RuntimeWarning: invalid value er
/Users/anaconda3/lib/python3.7/site-packages/ipykernel_launcher.py:64: RuntimeWarning: divide by zero e
/Users/anaconda3/lib/python3.7/site-packages/ipykernel_launcher.py:67: RuntimeWarning: divide by zero e
/Users/anaconda3/lib/python3.7/site-packages/ipykernel_launcher.py:68: RuntimeWarning: invalid value er
```

```
ValueError                                Traceback (most recent call last)
```

```
<ipython-input-141-5e79a106caa4> in <module>
```

```
10 #print(result)
11
--> 12 gmm._ll_joint(y,result[0], result[1], result[2])
13
14 y[:,0]
```

```
<ipython-input-139-f31eb59f7ff7> in _ll_joint(self, points, pi, mu, sigma, **kwargs)
```

```
70
71         # Posterior Distribution using Bayes Rule
--> 72         self.tau[:,i] = pi[i] * mvn
73
74         # normalize across columns to make a valid probability
```

```
ValueError: could not broadcast input array from shape (3,3,3) into shape (6)
```

In this section, you will use your GMM algorithm to do pixel clustering.

```
[ ] # helper function for plotting images. You don't have to modify it
def cluster_pixels_gmm(image, K):
    """Clusters pixels in the input image

    Args:
        image: input image of shape(H, W, 3)
        K: number of components

    Return:
        clustered_img: image of shape(H, W, 3) after pixel clustering
    """
    im_height, im_width, im_channel = image.shape
    flat_img = np.reshape(image, [-1, im_channel]).astype(np.float32)
    gamma, (pi, mu, sigma) = GMM()(flat_img, K=K, max_iters=100)
    cluster_ids = np.argmax(gamma, axis=1)
    centers = mu

    gmm_img = np.reshape(centers[cluster_ids], (im_height, im_width, im_channel))

    return gmm_img
```

```
[ ] # helper function for plotting images. You don't have to modify it
image = imageio.imread(imageio.core.urlopen(url).read())

gmm_image_5 = cluster_pixels_gmm(image, 5)
plot_images([image, gmm_image_5], ['origin', 'gmm=5'])

gmm_image_15 = cluster_pixels_gmm(image, 15)
plot_images([image, gmm_image_15], ['origin', 'gmm=15'])
```

```
NotImplementedError                                Traceback (most recent call last)
```

```
<ipython-input-147-2e90c0510260> in <module>
2 image = imageio.imread(imageio.core.urlopen(url).read())
3
--> 4 gmm_image_5 = cluster_pixels_gmm(image, 5)
5 plot_images([image, gmm_image_5], ['origin', 'gmm=5'])
6
```

```

<ipython-input-146-1e655f8d5dfa> in cluster_pixels_gmm(image, K)
    11 im_height, im_width, im_channel = image.shape
    12 flat_img = np.reshape(image, [-1, im_channel]).astype(np.float32)
--> 13 gamma, (pi, mu, sigma) = GMM()(flat_img, K=K, max_iters=100)
    14 cluster_ids = np.argmax(gamma, axis=1)
    15 centers = mu

<ipython-input-145-648373e468d1> in __call__(self, points, K, max_iters, abs_tol, rel_tol, **kwargs)
    74 Hint: You do not need to change it. For each iteration, we process E and M steps, then
    75 """
--> 76 pi, mu, sigma = self._init_components(points, K, **kwargs)
    77 pbar = tqdm(range(max_iters))
    78 for it in pbar:

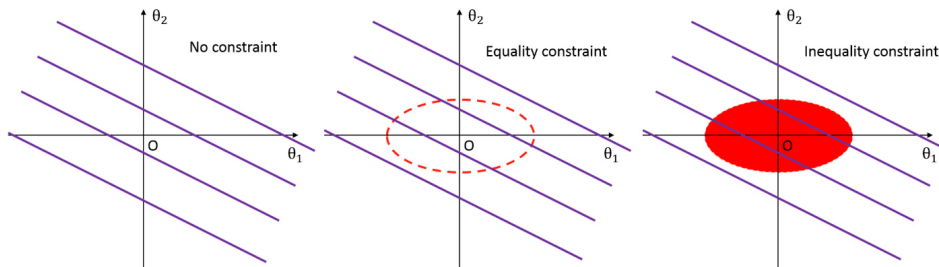
<ipython-input-145-648373e468d1> in _init_components(self, points, K, **kwargs)
    17 For instance, you could use ids, mu = KMeans()(points, K) to initialize.
    18 """
--> 19 raise NotImplementedError
    20
    21 def _ll_joint(self, points, pi, mu, sigma, **kwargs):

NotImplementedError:

```

4 Lagrangian [25 + 5 pts]

Assume in a coordinate (θ_1, θ_2) as shown in the image below, for any line that has a slope $k = 0.5$, just as the blue lines in this image, we want to find the possible minimum θ_2 -intercept of this line. Without a constraint, this minimum θ_2 -intercept can go to negative infinity (We just keep moving this line downward). However, if a constraint is added, which requires that this line has to fall on the red ellipse (Analytical form of this area is $\frac{\theta_1^2}{2^2} + \frac{\theta_2^2}{1^2} \leq 1$), this question will have a unique, bounded solution.



The above question can be mathematically formulated as a constrained optimization problem:

$$\begin{aligned}
 & \min_{\theta_1, \theta_2} f(\theta_1, \theta_2), \quad \text{where } f(\theta_1, \theta_2) = \theta_1 + 2\theta_2 \\
 & \text{subject to } g(\theta_1, \theta_2) \leq 0, \quad \text{where } g(x) = \theta_1^2 + 4\theta_2^2 - 4
 \end{aligned}$$

4.1 Write out the Lagrangian function. [5pts]

The Primal problem and the Dual problem are defined as,

Primal problem (The original optimization problem.):

$$\min_{\theta_1, \theta_2} \Theta_{\square}(\theta_1, \theta_2) = \min_{\theta_1, \theta_2} \max_{\lambda \geq 0} f(\theta_1, \theta_2) + \lambda(\theta_1^2 + 4\theta_2^2 - 4)$$

Dual problem (By shifting the order of *min* and *max* in the Primal problem.):

$$\max_{\lambda \geq 0} \Theta_{\square}(\lambda) = \max_{\lambda \geq 0} \min_{\theta_1, \theta_2} f(\theta_1, \theta_2) + \lambda(\theta_1^2 + 4\theta_2^2 - 4)$$

For this problem, the optimal solution to the Primal and the Dual problems are the same, which is often referred as strong duality. The optimal solution can be found out by meeting the KKT condition.

4.2 Write out the KKT conditions of the Lagrangian defined in 4.1. [10 pts]

KKT conditions include:

- (1). $\nabla_{\theta_1, \theta_2} \square = 0$. (Satisfied at the minimum of f .)
- (2). $\lambda g(\theta_1, \theta_2) = 0$. and (3). $\lambda \geq 0$. (Therefore, when this condition is met, $\square = f$.)
- (3). $g(\theta_1, \theta_2) \leq 0$. (The inequality constraint.)

4.3 Solve the optimization problem. [10pts]

(Hint: The optimal solution can be found either by solving the KKT conditions or by solving the Dual problem.)

4.4 Bonus [5 pts]

Will the solution be the same if an equality constraint

$$\frac{\theta_1^2}{2^2} + \frac{\theta_2^2}{1^2} = 1$$

is applied on this question? (The constraint is as shown in the picture below)

Please prove it mathematically.

4.1 Lagrangian functions:

$$L(\theta_1, \theta_2, \lambda) = \theta_1 + 2\theta_2 + \lambda(\theta_1^2 + 4\theta_2^2 - 4), \text{ where } f(\theta_1, \theta_2) = \theta_1 + 2\theta_2 \text{ and } g(\theta_1, \theta_2) = \theta_1^2 + 4\theta_2^2 - 4.$$

4.2 KKT Conditions:

$$(1) \frac{\partial L}{\partial \theta_1} = 0 \Rightarrow 1 + 2\lambda\theta_1 = 0$$

$$\frac{\partial L}{\partial \theta_2} = 0 \Rightarrow 2 + 8\lambda\theta_2 = 0$$

$$(2) \lambda(\theta_1^2 + 4\theta_2^2 - 4) = 0 \text{ and } \lambda \geq 0$$

$$(3) \theta_1^2 + 4\theta_2^2 - 4 \leq 0$$

4.3

$$(1) \theta_1 = -\frac{1}{2\lambda}$$

$$\theta_2 = -\frac{1}{4\lambda}$$

Plug in (2), we have

$$(2) \lambda(\theta_1^2 + 4\theta_2^2 - 4) = 0 \text{ and } \lambda \geq 0, \text{ solves to } \lambda_1 = \sqrt{\frac{1}{8}} \text{ and } \lambda_2 = 0, \text{ which is trivial solution.}$$

$$(3) \theta_1^2 + 4\theta_2^2 - 4 \leq 0 \Rightarrow \lambda \geq \sqrt{\frac{1}{8}}$$

$$\text{Plug back in (1), we get } \theta_1 = -\sqrt{2}, \theta_2 = -\frac{\sqrt{2}}{2}, f(\theta_1, \theta_2) = -2\sqrt{2}, \lambda = \sqrt{\frac{1}{8}}$$

4.4

Yes, the solution will be the same. Let's rewrite the Lagrangian function: $L(\theta_1, \theta_2, \lambda) = \theta_1 + 2\theta_2 + \lambda(\frac{\theta_1^2}{2^2} + \frac{\theta_2^2}{1^2} - 1)$, where $f(\theta_1, \theta_2) = \theta_1 + 2\theta_2$ and $g(\theta_1, \theta_2) = \frac{\theta_1^2}{2^2} + \frac{\theta_2^2}{1^2} - 1$.

KKT conditions thus become:

$$(1) \frac{\partial L}{\partial \theta_1} = 0 \Rightarrow 1 + \frac{2\lambda\theta_1}{4} = 0$$

$$\frac{\partial L}{\partial \theta_2} = 0 \Rightarrow 2 + 2\lambda\theta_2 = 0$$

$$(2) \lambda(\frac{\theta_1^2}{2^2} + \frac{\theta_2^2}{1^2} - 1) = 0 \text{ and } \lambda \geq 0$$

$$(3) \frac{\theta_1^2}{2^2} + \frac{\theta_2^2}{1^2} - 1 \leq 0$$

Now by plugging in $\theta_1 = -\frac{2}{\lambda}$ and $\theta_2 = -\frac{1}{\lambda}$, we obtain $\lambda = \sqrt{2}$, $\theta_1 = -\sqrt{2}$, $\theta_2 = -\frac{1}{\sqrt{2}}$, $F(\theta_1, \theta_2) = -2\sqrt{2}$. λ is different, but θ_1 and θ_2 is still the same.

Intuition: In 4.3 we want to find a minimum θ_2 intercept in the plane with the line fall on the red ellipse, where the ellipse can be described by $\theta_1^2 + 4\theta_2^2 - 4 \leq 0$. If we change less than to equal, the only thing that changes is that now the line has to fall on the edge of the red ellipse instead of anywhere on the ellipse. In the case of finding minimum intercept, however, we want the intercept to be as negative as possible, so one can easily understand the place where the line lies is the same.